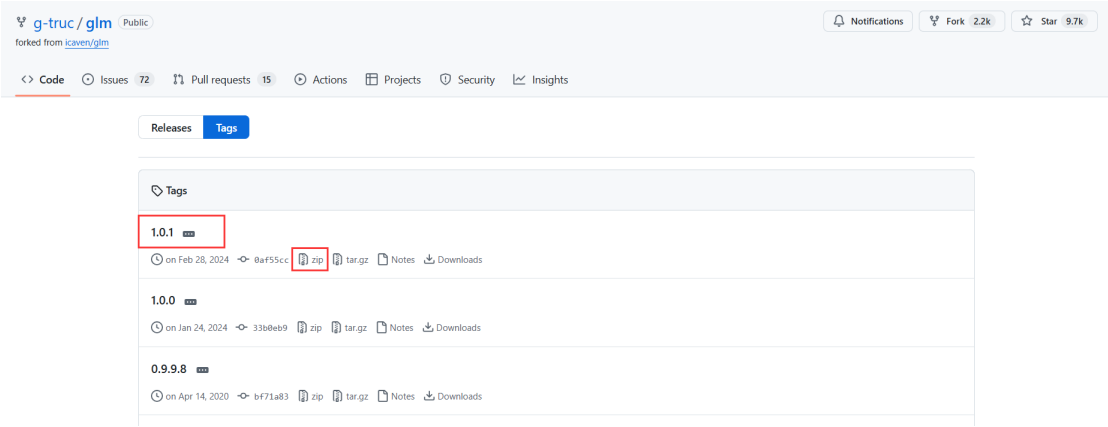
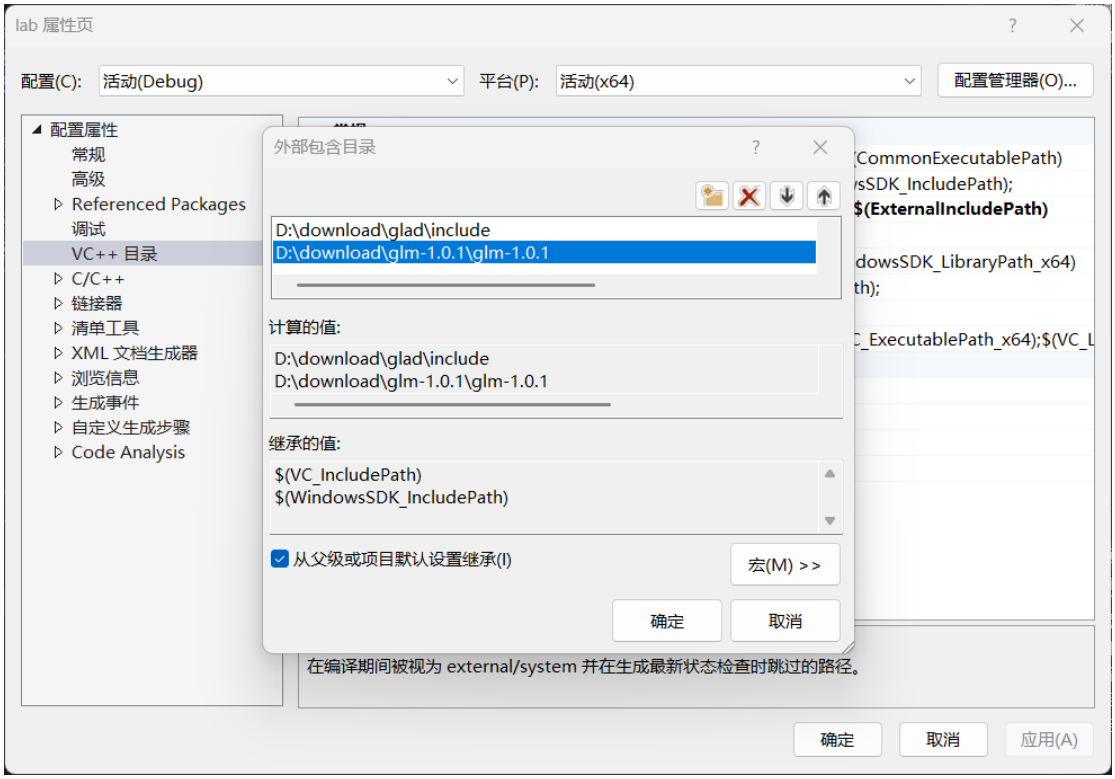
计算机图形学实验二——变换与坐标系统

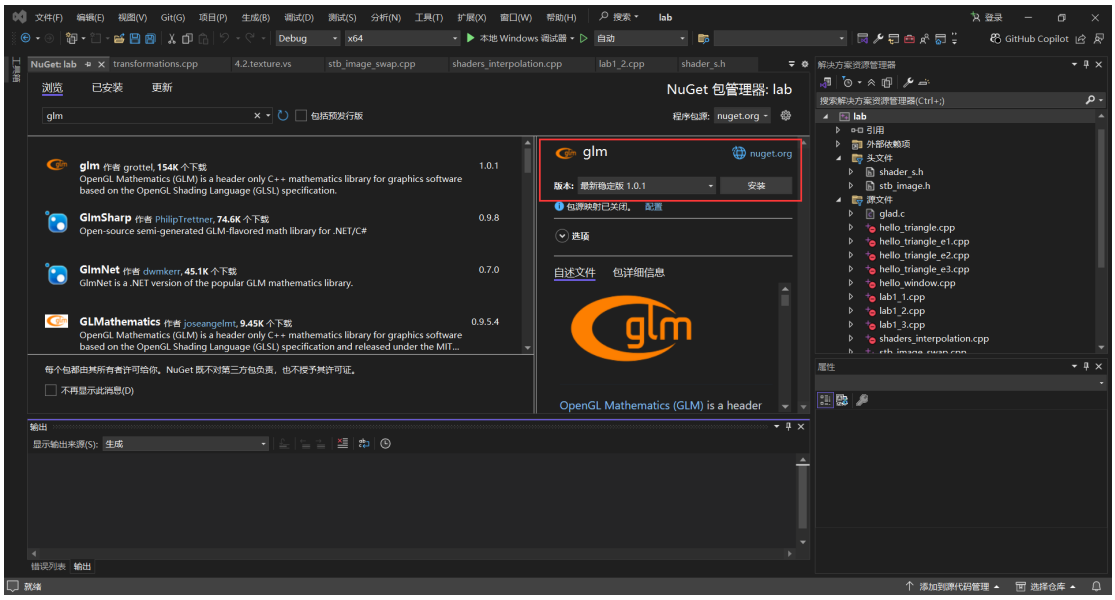
1. 安装 glm 库（教程 0.9.8 版，最新 1.0.1，下文用最新版），流程与 glad 相同
   https://glm.g-truc.net/0.9.8/index.html
   https://github.com/g-truc/glm/tags



根目录即为 glm-1.0.1

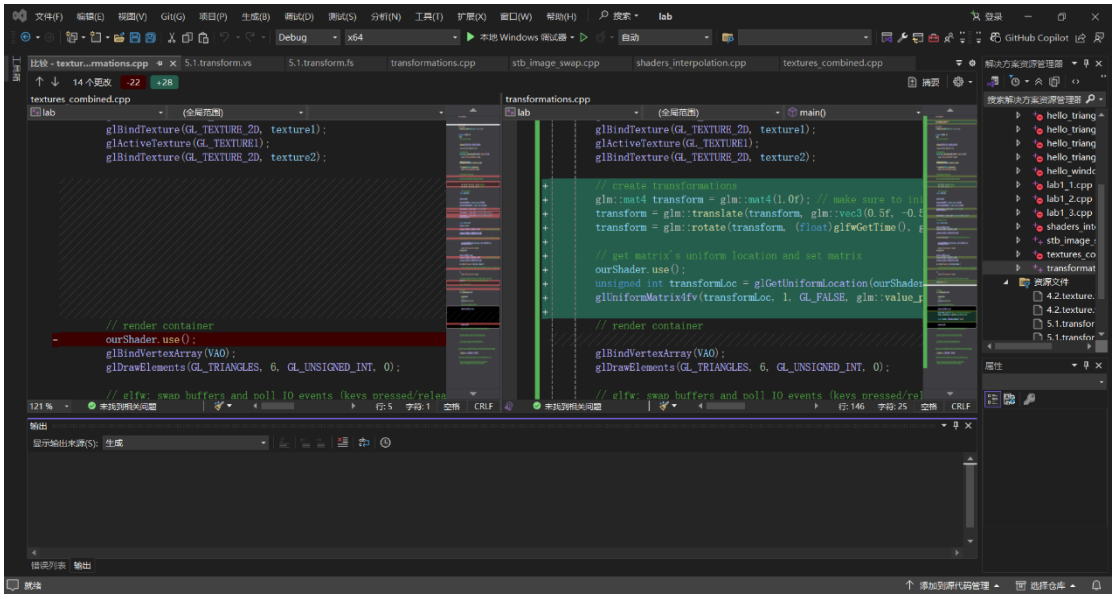


除了从官网下载，也可以使用 NuGeT 程序包管理器下载 glm



2. **阅读变换一节，运行示例程序“transformations.cpp”**
   https://learnopengl.com/code_viewer_gh.php?code=src/1.getting_started/5.1.transformations/transformations.cpp

   将” transformations.cpp” 与“textures_combined.cpp”对比，主要区别有：增加了 glm 变换相关函数，着色器的修改。



3. 在实验一的基础上，为立方体增加旋转变换
   着色器代码：

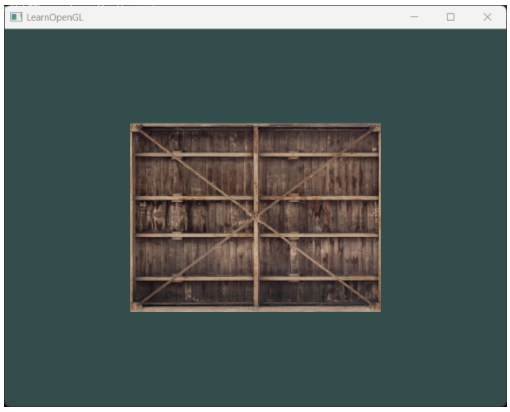
```
const char* vertexShaderSource = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"layout (location = 1) in vec3 aColor;\n"
"layout (location = 2) in vec2 aTexCoord;\n"
"out vec3 ourColor;\n"
```

```
"out vec2 TexCoord;\n"
"uniform mat4 transform;\n"
"void main()\n"
"{\n"
"    gl_Position = transform * vec4(aPos, 1.0);\n"
"    ourColor = aColor;\n"
"    TexCoord = vec2(aTexCoord.x, aTexCoord.y);\n"
"}\0";
```

修改变换矩阵，更改旋转轴，依次显示三个不同视角的视图。注意，在图像坐标系中，y 轴以向上为正，x 轴以向右为正。

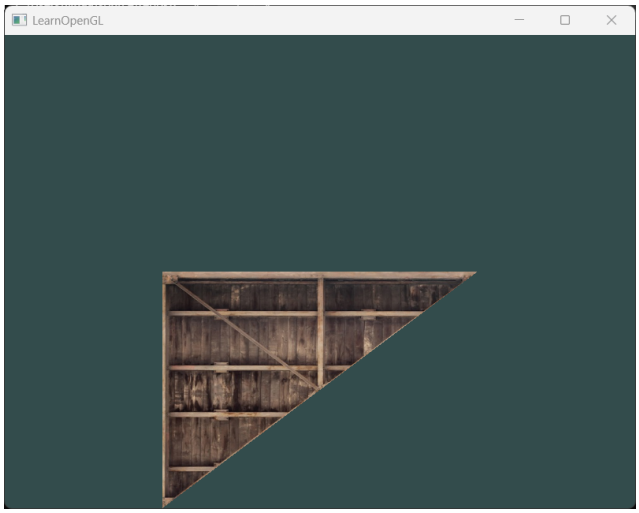
正视图（不旋转）：

```
glm::mat4 transform = glm::mat4(1.0f);
```



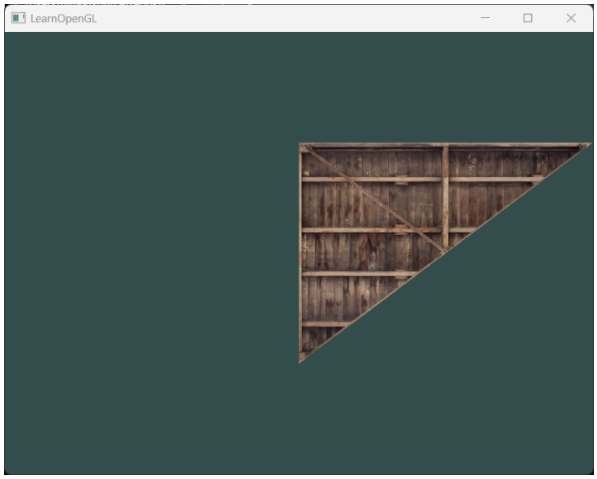
俯视图（绕 x 轴顺时针旋转 90 度）：

```
glm::mat4 transform = glm::mat4(1.0f);
transform = glm::rotate(transform, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
```



左视图（绕 y 轴顺时针旋转 90 度）：

```
glm::mat4 transform = glm::mat4(1.0f);
transform = glm::rotate(transform, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
```

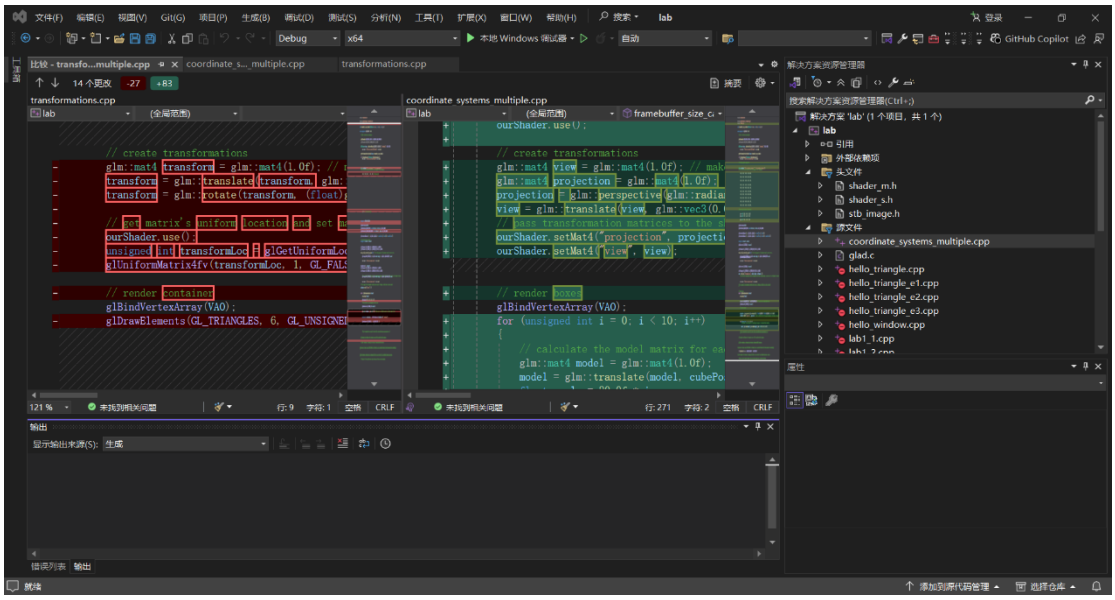


我们可以根据 vertices 与 indices 手动画出构建的立方体，并与绘制的三视图对比是否符合结果。

4. 阅读坐标系统一节，运行示例程序“coordinate_systems_multiple.cpp”

我们也可以先尝试运行之前的“coordinate_systems.cpp”和“coordinate_systems_depth.cpp”，它们的改动并不多。

将“coordinate_systems_multiple.cpp”与”transformations.cpp”对比，主要有以下区别：更改了 vertices，删去 indices 数组及关联的 EBO，增加了每个立方体的位置数组，相对应的更改了绘制方法；增加了深度测试设置，包括更改 glCLear 设置；更改了变换矩阵，增加了透视投影与多个立方体的变换方法；修改了着色器。



5. 为第 3 步的立方体设置透视投影。

根据教程与代码分析，设置透视投影需要处理变换矩阵及相应的着色器。

着色器代码：

```
const char* vertexShaderSource = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
```

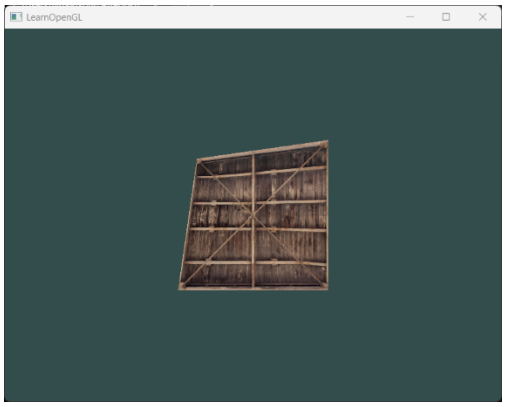
```
"layout (location = 1) in vec3 aColor;\n"
"layout (location = 2) in vec2 aTexCoord;\n"
"out vec3 ourColor;\n"
"out vec2 TexCoord;\n"
"uniform mat4 transform;\n"
"uniform mat4 model;\n"
"uniform mat4 view;\n"
"uniform mat4 projection;\n"

"void main()\n"
"{\n"
"    gl_Position = projection * view * model * transform * vec4(aPos, 1.0f);\n"
"    ourColor = aColor;\n"
"    TexCoord = vec2(aTexCoord.x, 1.0 - aTexCoord.y);\n"
"}\0";
```

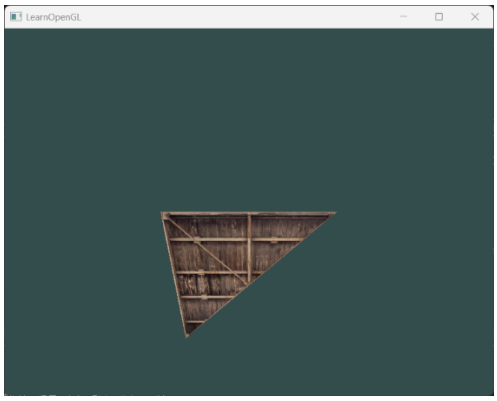
变换矩阵：

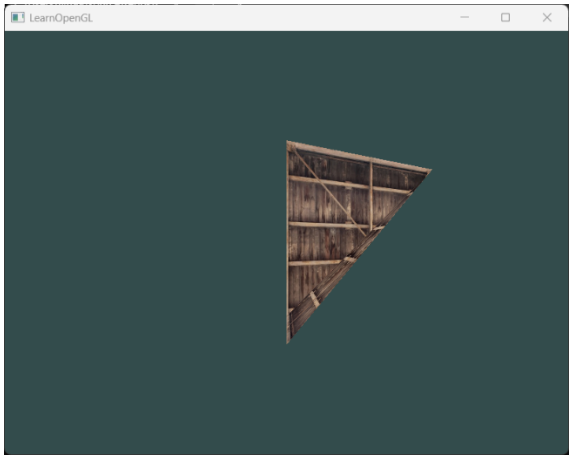
```
        glm::mat4 view = glm::mat4(1.0f); // make sure to initialize matrix to identity
matrix first
        glm::mat4 projection = glm::mat4(1.0f);
        projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH /
(float)SCR_HEIGHT, 0.1f, 100.0f);
        view = glm::translate(view, glm::vec3(0.0f, 0.0f, -3.0f));
        // pass transformation matrices to the shader

        // calculate the model matrix for each object and pass it to shader before
drawing
        glm::mat4 model = glm::mat4(1.0f);
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));

        // create transformations
        glm::mat4 transform = glm::mat4(1.0f); // make sure to initialize matrix to
identity matrix first
        transform = glm::rotate(transform, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f,
0.0f));
```

```
        unsigned int transformLoc = glGetUniformLocation(shaderProgram, "transform");
        glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
        unsigned int modelLoc = glGetUniformLocation(shaderProgram, "model");
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        unsigned int projectionLoc = glGetUniformLocation(shaderProgram, "projection");
        glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));
        unsigned int viewLoc = glGetUniformLocation(shaderProgram, "view");
        glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
```

绘制三视图，分别得到：







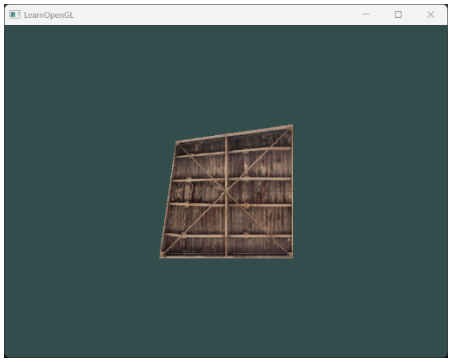
6. 为第 3 步的立方体设置深度测试。

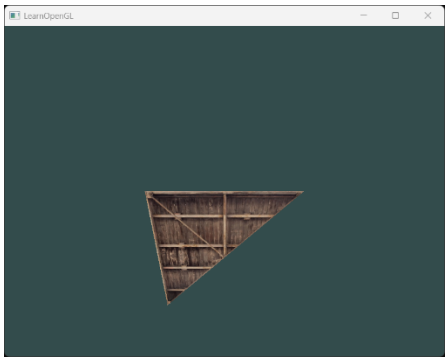
深度测试设置：

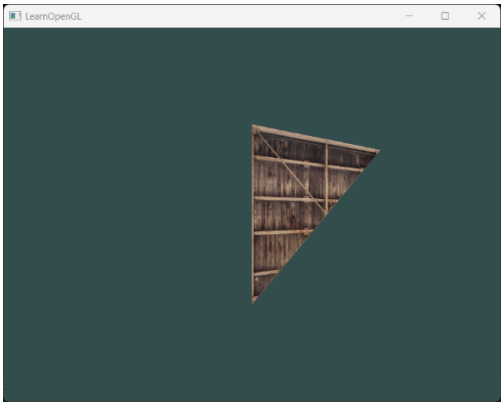
```
glEnable(GL_DEPTH_TEST);
```

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

绘制三视图，分别得到：







结果与第 5 步相同。这是由于我们设置每个面的连接顶点纹理一致导致的（即 indices 中在不同三角面中重复使用相同的点），可以通过像“coordinate_systems_multiple.cpp”中的 vertices 数组一样，对不同面的同一顶点设置不同纹理或颜色来体现差别。